

UNIT — 2 : CLASSES, OBJECTS, METHODS & WRAPPER CLASSES

Java Programming — Complete Unit 2 Notes

Course: Object-Oriented Programming through Java | **Year/Sem:** 2nd Year Sem 1 (2-1)

Syllabus Coverage: Wrapper Classes, Autoboxing & Unboxing, Method Definitions, Overloaded Methods, Overloaded Constructors, Passing Class Objects as Parameters, Access Control, Recursive Methods, Nesting of Methods, Final & Static Attributes.

Source: ChatGPT Curated Exam Preparation Notes

Below are **definition + syntax + real-time Java program + output** for each topic.

1. Wrapper Classes

Definition

A **Wrapper Class** converts a primitive data type into an object.

Primitive	Wrapper Class
int	Integer
float	Float
double	Double
char	Character
boolean	Boolean
byte	Byte
short	Short
long	Long

Syntax

```
Integer obj = Integer.valueOf(100);
```

Real-time program

Suppose an online shopping application stores product quantity in an **ArrayList**. **ArrayList** stores objects, so **Integer** is used.

```
import java.util.ArrayList;

class WrapperExample {
    public static void main(String[] args) {

        ArrayList<Integer> quantities = new ArrayList<>();

        quantities.add(5);
        quantities.add(10);
        quantities.add(15);

        System.out.println(quantities);
    }
}
```

Output:

```
[5, 10, 15]
```

2. Autoboxing

Definition

Autoboxing is the automatic conversion of a primitive value into its corresponding wrapper object.

Syntax

```
int n = 10;  
Integer obj = n;
```

Real-time program

```
class AutoboxingExample {  
    public static void main(String[] args) {  
  
        int price = 500;  
  
        // int automatically converted to Integer  
        Integer productPrice = price;  
  
        System.out.println(productPrice);  
    }  
}
```

Output:

500

Here:

int → Integer

3. Unboxing

Definition

Unboxing is the automatic conversion of a wrapper object into its corresponding primitive value.

Syntax

```
Integer obj = 100;  
int n = obj;
```

Real-time program

```
class UnboxingExample {  
    public static void main(String[] args) {  
  
        Integer productPrice = 500;  
  
        // Integer automatically converted to int  
        int price = productPrice;  
  
        System.out.println(price);  
    }  
}
```

Output:

500

Here:

Integer → int

Easy difference

Autoboxing : int → Integer
Unboxing : Integer → int

4. Introduction to Methods

Definition

A **method** is a block of code that performs a specific task. Methods help us **reuse code**.

Syntax

```
returnType methodName(parameters) {  
    // statements  
}
```

Real-time program

A banking application can use a method to calculate the account balance.

```
class Bank {  
  
    static int calculateBalance(int deposit, int withdrawal) {  
        return deposit - withdrawal;  
    }  
  
    public static void main(String[] args) {  
  
        int balance = calculateBalance(10000, 3000);  
  
        System.out.println("Balance = " + balance);  
    }  
}
```

Output:

```
Balance = 7000
```

5. Defining Methods

Definition

Defining a method means writing the method name, parameters, return type and statements that perform the required operation.

Syntax

```
returnType methodName(parameter1, parameter2) {  
    // method body  
    return value;  
}
```

Example

```
class Student {  
  
    static int totalMarks(int m1, int m2, int m3) {  
        return m1 + m2 + m3;  
    }  
  
    public static void main(String[] args) {  
  
        int total = totalMarks(80, 85, 90);  
  
        System.out.println("Total Marks = " + total);  
    }  
}
```

Output:

```
Total Marks = 255
```

6. Overloaded Methods

Definition

Method overloading means having multiple methods with the **same name but different parameter lists**.

The difference can be in:

- Number of parameters
- Type of parameters
- Order of parameters

Syntax

```
add(int a, int b)
add(int a, int b, int c)
```

Real-time program

A calculator can perform addition of two or three numbers.

```
class Calculator {

    static int add(int a, int b) {
        return a + b;
    }

    static int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {

        System.out.println("Two numbers = " + add(10, 20));

        System.out.println("Three numbers = " + add(10, 20, 30));

    }
}
```

Output:

```
Two numbers = 30
Three numbers = 60
```

Important

This is **not** valid overloading:

```
int add(int a, int b)
double add(int a, int b)
```

Changing only the return type is **not method overloading**.

7. Overloaded Constructor Methods

Definition

Constructor overloading means having multiple constructors in the same class with different parameter lists.

Constructors are used to initialize objects.

Syntax

```
class Student {  
  
    Student() {  
    }  
  
    Student(String name) {  
    }  
  
    Student(String name, int age) {  
    }  
}
```

Real-time program

A student management system can create students with different available information.

```
class Student {  
  
    String name;  
    int age;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
  
    Student(String name) {  
        this.name = name;  
        age = 18;  
    }  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    void display() {  
        System.out.println(name + " - " + age);  
    }  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
        Student s2 = new Student("Abhiram");  
        Student s3 = new Student("Rahul", 20);  
  
        s1.display();  
        s2.display();  
        s3.display();  
    }  
}
```

Output:

```
Unknown - 0  
Abhiram - 18  
Rahul - 20
```

8. Class Objects as Parameters in Methods

Definition

In Java, an **object of a class** can be passed as a parameter to a method.

Syntax

```
void methodName(ClassName obj) {  
    // statements  
}
```

Real-time program

Compare marks of two students.

```
class Student {  
  
    String name;  
    int marks;  
  
    Student(String name, int marks) {  
        this.name = name;  
        this.marks = marks;  
    }  
  
    void compare(Student s) {  
  
        if (this.marks > s.marks) {  
            System.out.println(this.name + " has higher marks");  
        } else {  
            System.out.println(s.name + " has higher marks");  
        }  
    }  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student("Abhiram", 90);  
        Student s2 = new Student("Rahul", 85);  
  
        s1.compare(s2);  
    }  
}
```

Output:

Abhiram has higher marks

Here:

```
s1.compare(s2);
```

means object **s2** is passed as a parameter.

9. Access Control

Definition

Access control determines where a class member can be accessed.

Java has four important access levels:

Access modifier	Same class	Same package	Subclass	Other package
private	✓	✗	✗	✗
default	✓	✓	✗*	✗
protected	✓	✓	✓	✗*
public	✓	✓	✓	✓

* There are additional rules for subclasses in different packages.

Syntax

```
private int balance;  
public void display() {  
}
```

Real-time program

Bank balance should not be directly changed from outside the class.

```
class BankAccount {  
    private double balance = 10000;  
  
    public void showBalance() {  
        System.out.println("Balance = " + balance);  
    }  
  
    public void deposit(double amount) {  
        balance = balance + amount;  
    }  
  
    public static void main(String[] args) {  
        BankAccount account = new BankAccount();  
  
        account.deposit(5000);  
        account.showBalance();  
    }  
}
```

Output:

```
Balance = 15000.0
```

balance is **private**, so it is accessed through public methods.

10. Recursive Methods

Definition

A **recursive method** is a method that calls itself until a stopping condition, called the **base condition**, is reached.

Syntax

```
void method() {  
    if (condition) {  
        return;  
    }  
  
    method();  
}
```

Real-time program

Calculate factorial:

```
class Factorial {  
    static int factorial(int n) {  
        if (n == 0) {  
            return 1;  
        }  
  
        return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
        int result = factorial(5);  
  
        System.out.println("Factorial = " + result);  
    }  
}
```

Output:

```
Factorial = 120
```

Working

```
5 × factorial(4)
4 × factorial(3)
3 × factorial(2)
2 × factorial(1)
1 × factorial(0)

= 120
```

11. Nesting of Methods

Definition

Nesting of methods means one method calls another method to perform a task.

> Java does not normally allow defining a method directly inside another method. Instead, one method can **call** another method.

Syntax

```
method1() {
    method2();
}

method2() {
}
```

Real-time program

An online shopping system calculates the total using separate methods.

```
class Shopping {

    static int calculatePrice(int price, int quantity) {
        return price * quantity;
    }

    static void generateBill() {

        int total = calculatePrice(500, 3);

        System.out.println("Total Bill = " + total);
    }

    public static void main(String[] args) {

        generateBill();
    }
}
```

Output:

```
Total Bill = 1500
```

Here:

```
main()
  ↓
generateBill()
  ↓
calculatePrice()
```

12. final Attribute

Definition

A **final variable** is a variable whose value cannot be changed after it is initialized.

Syntax

```
final dataType variableName = value;
```

Real-time program

A company's GST rate or a fixed registration fee may be treated as a constant.

```
class Product {  
    final double GST = 18.0;  
  
    void display() {  
        System.out.println("GST = " + GST + "%");  
    }  
  
    public static void main(String[] args) {  
        Product p = new Product();  
        p.display();  
    }  
}
```

Output:

```
GST = 18.0%
```

This would cause an error:

```
GST = 20.0;
```

because **GST** is **final**.

13. static Attribute

Definition

A **static variable** belongs to the class rather than to individual objects. All objects share the same static variable.

Syntax

```
static dataType variableName;
```

Real-time program

A college can maintain one common college name for all students.

```
class Student {  
    String name;  
    static String college = "SRKR Engineering College";  
  
    Student(String name) {  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println(name + " - " + college);  
    }  
  
    public static void main(String[] args) {  
        Student s1 = new Student("Abhiram");  
        Student s2 = new Student("Rahul");  
  
        s1.display();  
        s2.display();  
    }  
}
```

Output:

Abhiram - SRKR Engineering College
Rahul - SRKR Engineering College

Both objects share the same **college** variable.

Quick Revision Table

Topic	Meaning	Example
Wrapper Class	Primitive represented as object	Integer
Autoboxing	Primitive → Wrapper	Integer x = 10;
Unboxing	Wrapper → Primitive	int x = obj;
Method	Performs a specific task	add()
Method Overloading	Same method name, different parameters	add(int, int)
Constructor Overloading	Multiple constructors with different parameters	Student() / Student(String)
Object as Parameter	Passing object to a method	compare(Student s)
Access Control	Controls member accessibility	private, public
Recursion	Method calls itself	factorial()
Method Nesting	One method calls another	generateBill() → calculatePrice()
final	Value cannot be changed	final int x = 10
static	Shared by all objects	static String college

Most important differences for exams

Autoboxing : **int → Integer**
Unboxing : **Integer → int**

Overloading : **Same name + different parameters**
Recursion : **Method calls itself**
static : **Shared by all objects**
final : **Cannot be changed**
private : **Accessible only inside the class**
public : **Accessible from anywhere (subject to normal Java access rules)**

Original ChatGPT Session Reference:

Shared Chat Link: <https://chatgpt.com/share/6ac3dc3e-5570-83ee-bea1-75b98a3ab26c>

Note: This document compiles the complete B.Tech / Degree 2nd Year 1st Semester (2-1) Java Programming notes for Unit-2, generated directly from the ChatGPT session at the above link.